

Команда 20

Кросс-валидация

Методы оценивания обобщающей способности
моделей машинного обучения

Основы машинного обучения. 2026



Содержание

- 1** Постановка задачи
- 2** Программная реализация
- 3** Численный пример
- 4** Альтернативные схемы и их сравнение

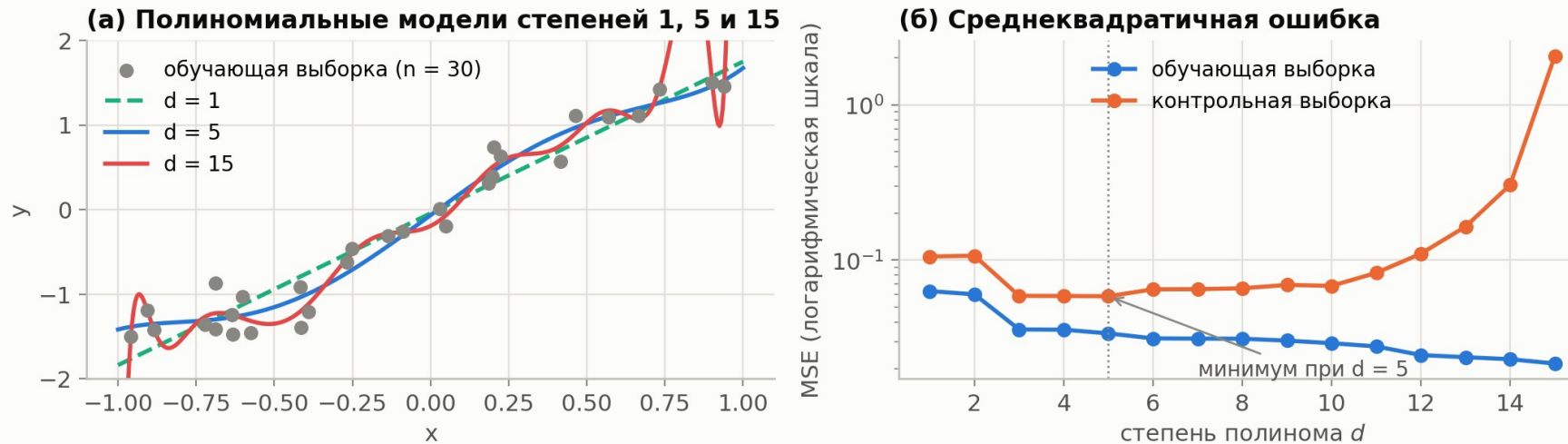
Все численные результаты, приведённые далее, получены программно; исходный код и рисунки находятся в каталоге src.

1

Постановка задачи

Эмпирическая ошибка и обобщающая способность

Рис. 1. Эмпирическая ошибка и ошибка на контрольной выборке в зависимости от сложности модели



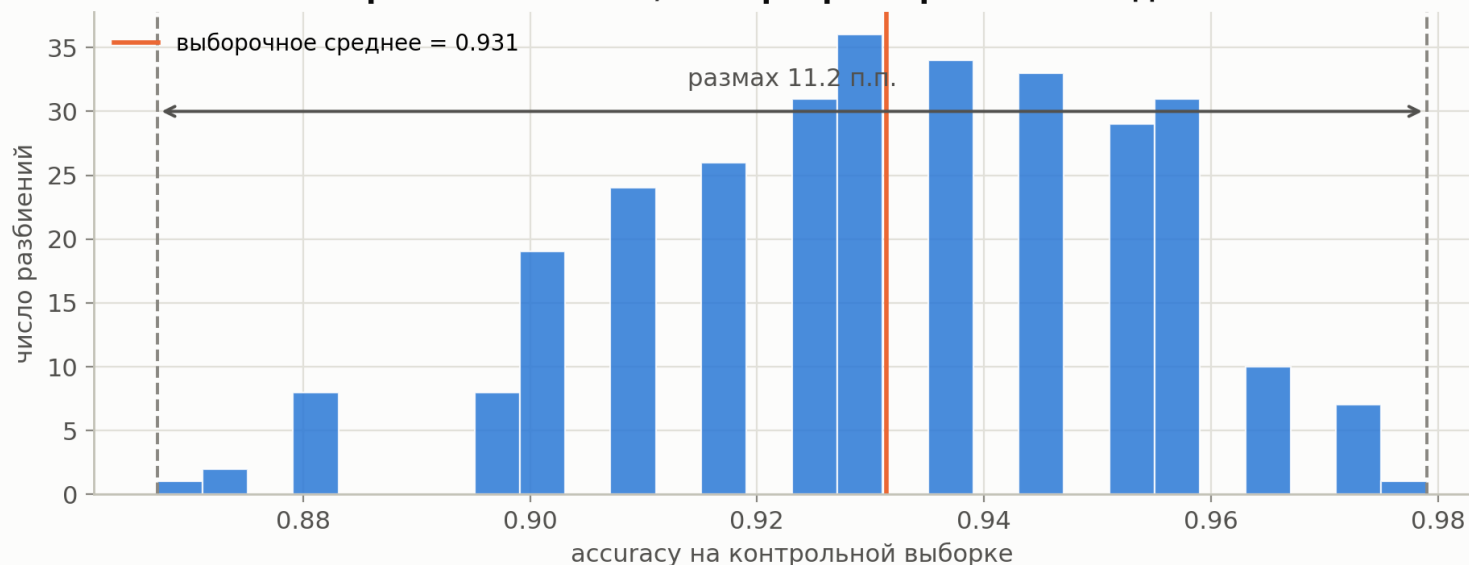
```
for d in range(1, 16):  
    model = make_pipeline(PolynomialFeatures(d), LinearRegression())  
    model.fit(X_train, y_train)  
    err_train.append(mean_squared_error(y_train, model.predict(X_train)))  
    err_test.append(mean_squared_error(y_test, model.predict(X_test)))
```

- $d = 5$: $MSE = 0.055$ — минимум на контрольной выборке
- $d = 15$: $MSE = 0.022$ на обучающей, 2.063 на контрольной
- отношение $2.063 / 0.059 \approx 35$

Эмпирическая ошибка монотонно убывает с ростом сложности модели, поэтому не может служить критерием выбора: её минимум достигается на модели с наихудшей обобщающей способностью.

Дисперсия оценки при однократном разбиении

Рис. 2. Распределение оценки accuracy по 300 случайным разбиениям train/test при фиксированной модели



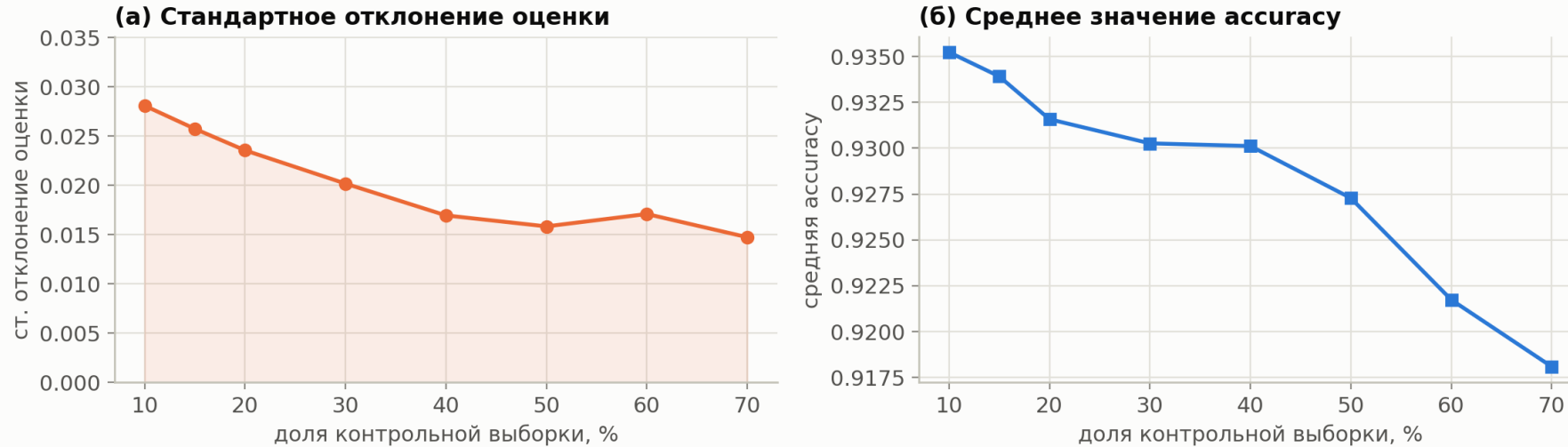
```
scores = []
for seed in range(300):
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.25, random_state=seed, stratify=y)
    model.fit(X_tr, y_tr)
    scores.append(accuracy_score(y_te, model.predict(X_te)))
```

- минимум 0.8671, максимум 0.9790
- размах 11.2 процентного пункта
- **5-блочная кросс-валидация: 0.9227 ± 0.0295**

Модель и выборка фиксированы, изменяется только значение `random_state`. Оценка по одному разбиению является реализацией случайной величины, дисперсия которой превышает типичные различия между моделями.

Выбор размера контрольной выборки

Рис. 3. Зависимость стандартного отклонения оценки и среднего качества модели от доли контрольной выборки (120 повторов на точку)



```
for share in [0.10, 0.15, 0.20, 0.30, 0.50, 0.70]:  
    vals = []  
    for seed in range(120):  
        vals.append(score_one_split(X, y, share, seed))  
    stds.append(np.std(vals))      # устойчивость оценки  
    means.append(np.mean(vals))   # качество модели
```

- доля 10 %: std = 0.0281, accuracy = 0.9352
- доля 70 %: std = 0.0147, accuracy = 0.9181
- оптимума у этого компромисса нет

Увеличение контрольной выборки снижает дисперсию оценки и одновременно ухудшает саму модель. Кросс-валидация устраняет противоречие: контролю подвергается каждый объект, а обучение всякий раз ведётся на $(K-1)/K$ выборки.

К-блочная кросс-валидация: определение и реализация



$$CV(K) = (1/K) \cdot \sum L(f_{-k}, D_k), \quad k = 1 \dots K$$

f_{-k} — модель, обученная без блока k ; D_k — блок k

```
def kfold_cross_val(estimator, X, y, K=5, random_state=0):  
    idx = np.arange(len(y))  
    np.random.RandomState(random_state).shuffle(idx)  
    folds = np.array_split(idx, K)  
  
    scores = []  
    for k in range(K):  
        test = folds[k]  
        train = np.concatenate([folds[j] for j in range(K)  
                                if j != k])  
        estimator.fit(X[train], y[train])  
        pred = estimator.predict(X[test])  
        scores.append(accuracy_score(y[test], pred))  
    return np.array(scores)
```

- процедура оценивает среднюю ошибку модели, обученной на $(K-1)/K$ выборки;
- каждый объект ровно один раз входит в контрольный блок;
- наряду со средним вычисляется стандартное отклонение по блокам.

Приведённая реализация не использует сплиттеры scikit-learn; её результат совпадает с `cross_val_score(cv=KFold(...))` — максимальное расхождение 0.00e+00.

2

Программная реализация

Базовый интерфейс scikit-learn

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
model = make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=5000))
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_val_score(model, X, y, cv=cv, scoring='accuracy')
scores.mean(), scores.std()
```

```
array([0.9737, 0.9474,
       0.9649, 0.9912,
       0.9912])
```

```
(0.9737, 0.0166)
```

блок k = 1

0.9737

блок k = 2

0.9474

блок k = 3

0.9649

блок k = 4

0.9912

блок k = 5

0.9912

среднее ± стандартное отклонение

0.9737 ± 0.0166

Выборка breast_cancer: 569 объектов, 30 признаков, два класса. Результат сообщается как среднее и стандартное отклонение по блокам: без оценки разброса судить о значимости различий между моделями невозможно.

Функции `cross_validate` и `cross_val_predict`

```
res = cross_validate(model, X, y, cv=cv,  
                     scoring=['accuracy', 'precision', 'recall',  
                             'f1', 'roc_auc'],  
                     return_train_score=True, n_jobs=-1)  
  
res['test_accuracy'].mean()    # 0.9737  
res['train_accuracy'].mean()   # 0.9881  
res['fit_time'].mean()         # 0.0041 c
```

```
oof = cross_val_predict(model, X, y, cv=cv)  
  
# каждый объект предсказан моделью,  
# которая его не наблюдала  
confusion_matrix(y, oof)  
accuracy_score(y, oof)        # 0.9736  
  
# используется для анализа ошибок и стекинга
```

метрика	контрольная, среднее	стандартное отклонение	обучающая, среднее	разность
accuracy	0.9737	0.0166	0.9881	0.0145
precision	0.9683	0.0290	0.9861	0.0178
recall	0.9916	0.0112	0.9951	0.0035
f1	0.9794	0.0127	0.9906	0.0112
roc_auc	0.9953	0.0053	0.9977	0.0024

Параметр `return_train_score` даёт диагностику переобучения: разность оценок на обучающей и контрольной выборках не превышает 0.018 ни по одной из метрик.

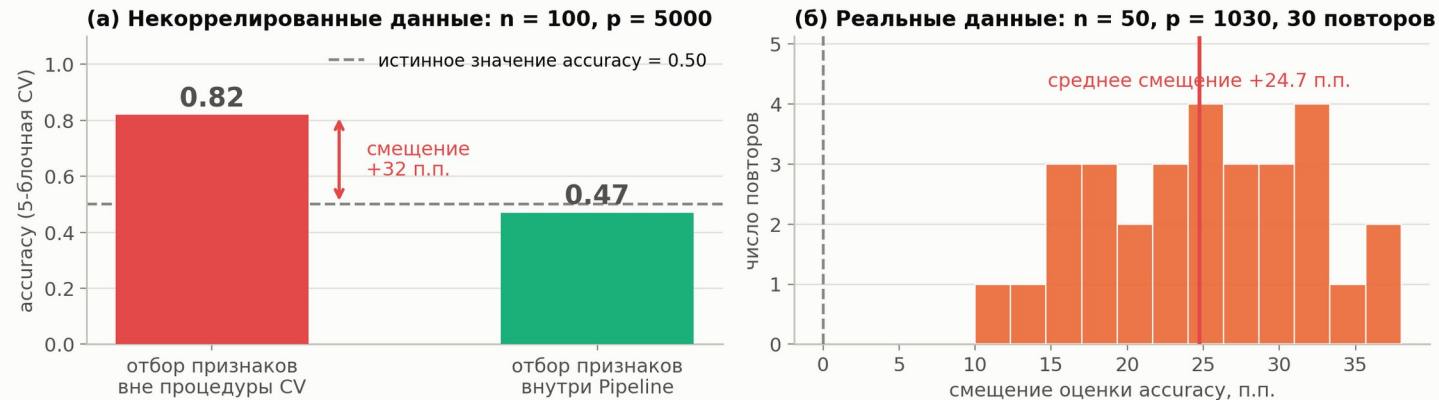
Смещение оценки при утечке информации

```
# предобработка выполнена до разбиения
X = StandardScaler().fit_transform(X)
X = SelectKBest(f_classif, k=20).fit_transform(X, y)
cross_val_score(model, X, y, cv=cv)

# отбор признаков использовал целевую переменную
# всех объектов, включая контрольные блоки
```

```
# предобработка входит в процедуру
pipe = Pipeline([
    ('scale', StandardScaler()),
    ('select', SelectKBest(f_classif, k=20)),
    ('clf', KNeighborsClassifier(3))])
cross_val_score(pipe, X, y, cv=cv)
# шаги переобучаются заново на каждом блоке
```

Рис. 5. Смещение оценки при отборе признаков вне процедуры кросс-валидации



На некоррелированных данных предобработка вне процедуры даёт оценку 0.82 при истинном значении 0.50. На реальных данных среднее смещение составило +24.7 п.п. и наблюдалось во всех 30 повторах.

Конвейер преобразований

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ('impute', SimpleImputer(strategy='median')),
    ('scale', StandardScaler()),
    ('select', SelectKBest(f_classif, k=10)),
    ('clf', LogisticRegression(max_iter=5000)),
])

scores = cross_val_score(pipe, X, y, cv=cv, scoring='roc_auc')

# шаги адресуются через двойное подчёркивание:
grid = {'select__k': [5, 10, 20], 'clf__C': [0.1, 1, 10]}
```

Преобразования, требующие включения в конвейер

StandardScaler, MinMaxScaler

оценивают среднее и дисперсию

SimpleImputer

оценивает медиану или среднее

SelectKBest, RFE

используют целевую переменную

PCA, TruncatedSVD

оценивают главные компоненты

TargetEncoder

оценивает средние по категориям

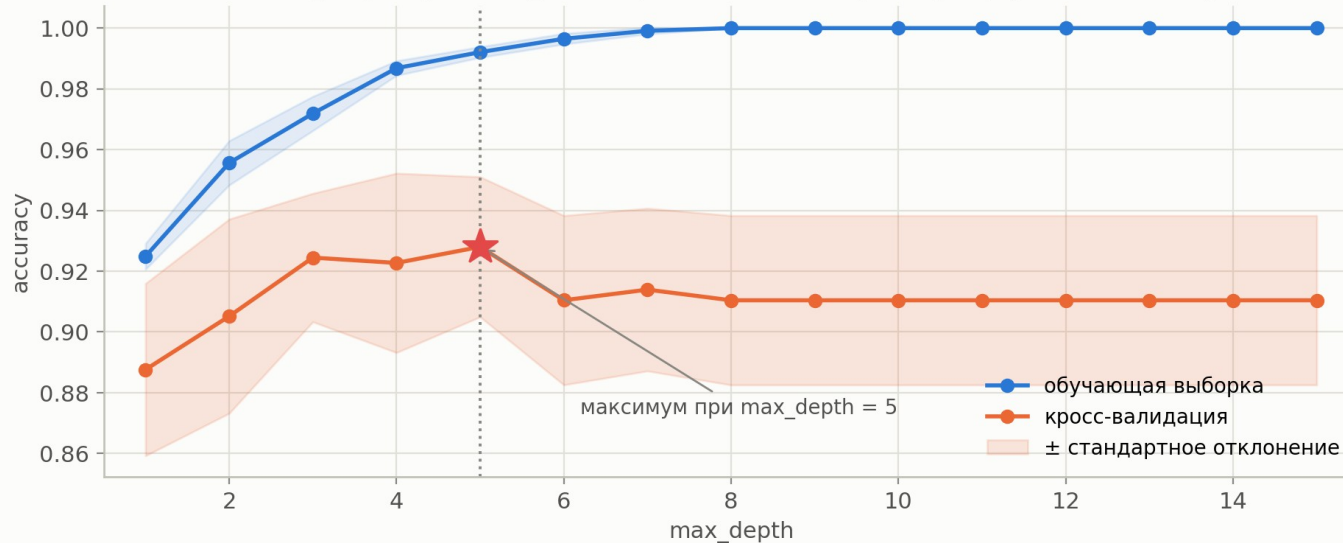
SMOTE и др. балансировка

порождает новые объекты

Внутри процедуры метод `fit` конвейера вызывается отдельно на каждом обучающем блоке, поэтому параметры преобразований оцениваются только по обучающим данным.

Подбор гиперпараметра: валидационная кривая

Рис. 6. Валидационная кривая решающего дерева по параметру `max_depth` (breast_cancer, 5-блочная стратифицированная CV)



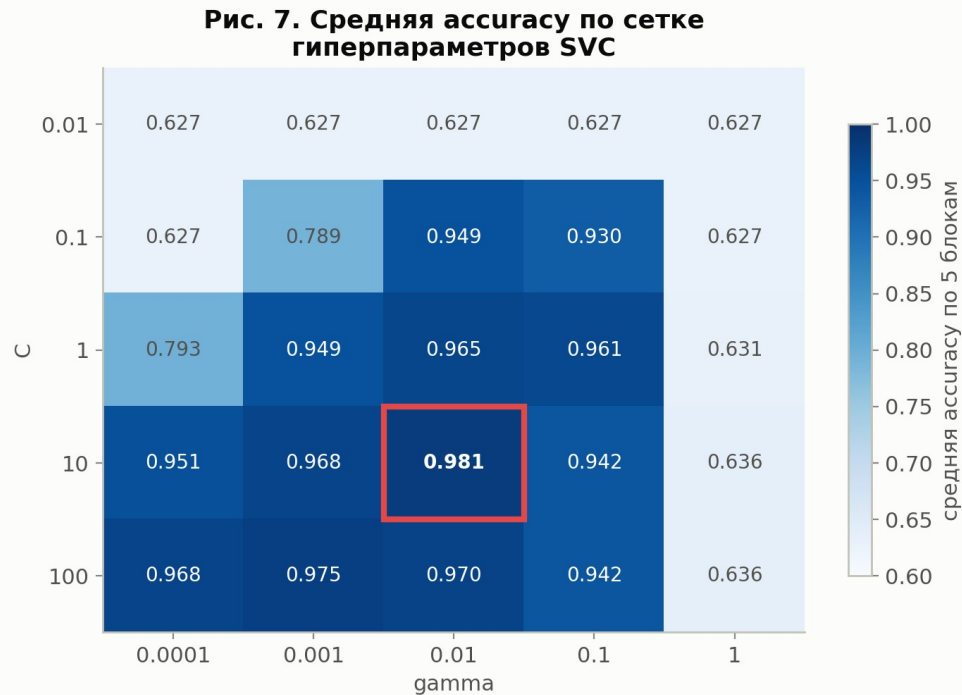
```
train, test = validation_curve(
    DecisionTreeClassifier(),
    X, y,
    param_name='max_depth',
    param_range=range(1, 16),
    cv=cv,
    scoring='accuracy',
    n_jobs=-1)

best = test.mean(axis=1).argmax()
```

- `max_depth` = 5: оценка 0.9280 — максимум
- `max_depth` = 15: 1.0000 на обучающей выборке и 0.9104 на контрольной
- ширина коридора $\pm$ std сопоставима с разностью между соседними значениями

Оценка на обучающей выборке достигает единицы и далее не изменяется, поэтому непригодна для выбора параметра; кросс-валидационная оценка имеет выраженный максимум.

Подбор конфигурации по сетке



```
pipe = Pipeline([('scale', StandardScaler()),
                  ('clf', SVC())])

grid = {'clf__C': [0.01, 0.1, 1, 10, 100],
        'clf__gamma': [1e-4, 1e-3, 1e-2, 0.1, 1]}

gs = GridSearchCV(pipe, grid, cv=cv,
                  scoring='accuracy', n_jobs=-1)

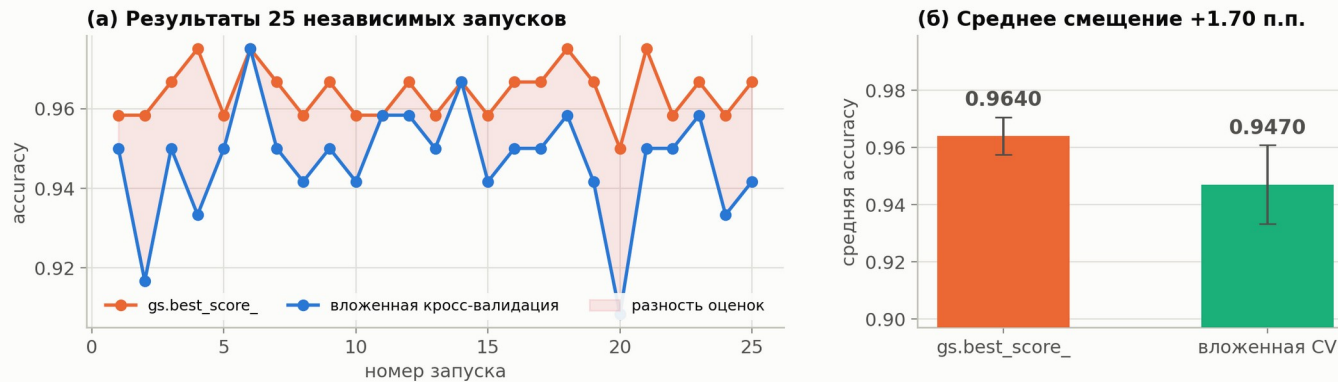
gs.fit(X, y)

gs.best_params_ # {'clf__C': 10, 'clf__gamma': 0.01}
gs.best_score_  # 0.9807
gs.best_estimator_ # переобучена на всей выборке
```

- 25 конфигураций × 5 блоков = 125 обучений;
- область высоких значений образует плато 0.94–0.98, следовательно решение устойчиво;
- изолированный максимум среди низких значений следует трактовать как случайный;
- при числе гиперпараметров более пяти применяется RandomizedSearchCV.

Вложенная кросс-валидация

Рис. 9. Оптимистическое смещение результата подбора гиперпараметров относительно вложенной кросс-валидации



```
inner = StratifiedKFold(4, shuffle=True)
outer = StratifiedKFold(4, shuffle=True)
```

```
gs = GridSearchCV(pipe, grid, cv=inner)
sc = cross_val_score(gs, X, y, cv=outer)
```

```
gs.best_score_ # выбор конфигурации
sc.mean()      # оценка качества
```

- внутренний цикл выбирает конфигурацию;
- внешний оценивает процедуру целиком;
- число обучений растёт с $4 \times 36 = 144$ до $4 \times 4 \times 36 = 576$.

Величина `gs.best_score_` есть максимум по 36 оценкам, каждая из которых содержит случайную составляющую. Математическое ожидание максимума набора случайных величин превышает математическое ожидание каждой из них, поэтому такая оценка систематически смещена вверх и не может использоваться в качестве оценки качества модели.

Эксперимент: подвыборка 120 объектов, сетка из 36 конфигураций, 25 независимых запусков. Среднее смещение +1.70 п.п.; смещение вверх наблюдалось в 22 запусках из 25. При отсутствии настраиваемых гиперпараметров вложенная процедура не требуется.

3

Численный пример

Постановка задачи и шаги 1–3

Выборка: $y = (2, 4, 6, 8, 10, 12)$, $n = 6$. Модель: константный прогноз, равный среднему обучающей выборки. Функция потерь: MAE.

Требуется оценить среднюю абсолютную ошибку модели на новом наблюдении из той же совокупности.

Шаг 1. Выборка делится на $K = 3$ последовательных блока по два наблюдения: $D_1 = \{2, 4\}$, $D_2 = \{6, 8\}$, $D_3 = \{10, 12\}$

Шаг 2. Каждый блок поочерёдно объявляется контрольным; модель обучается на объединении остальных.

k = 1

обучение $\{6, 8, 10, 12\}$

$$f_{-1} = (6+8+10+12)/4 = 9$$

контроль $\{2, 4\}$

$$|2-9| = 7, \quad |4-9| = 5$$

$$L_1 = (7 + 5)/2 = 6$$

k = 2

обучение $\{2, 4, 10, 12\}$

$$f_{-2} = (2+4+10+12)/4 = 7$$

контроль $\{6, 8\}$

$$|6-7| = 1, \quad |8-7| = 1$$

$$L_2 = (1 + 1)/2 = 1$$

k = 3

обучение $\{2, 4, 6, 8\}$

$$f_{-3} = (2+4+6+8)/4 = 5$$

контроль $\{10, 12\}$

$$|10-5| = 5, \quad |12-5| = 7$$

$$L_3 = (5 + 7)/2 = 6$$

Шаг 3. Усреднение по блокам: $CV(3) = (6 + 1 + 6) / 3 = 13 / 3 \approx 4.33$

Оценки по блокам равны 6, 1 и 6 при среднем 4.33: стандартное отклонение сопоставимо со средним, что указывает на неоднородность блоков.

Влияние перемешивания на результат

Рис. 10. Разбиение на три блока и значения MAE при shuffle=False и shuffle=True

(a) shuffle=False: исходный (упорядоченный) порядок

	обучение		тест			
Блок 1	2	4	6	8	10	12
Блок 2	2	4	6	8	10	12
Блок 3	2	4	6	8	10	12
					прогноз = 9	MAE = 6
					прогноз = 7	MAE = 1
					прогноз = 5	MAE = 6
						CV = 4.33

(б) shuffle=True: порядок 2, 10, 4, 12, 6, 8

Блок 1	2	10	4	12	6	8
Блок 2	2	10	4	12	6	8
Блок 3	2	10	4	12	6	8
					прогноз = 7.5	MAE = 4
					прогноз = 6.5	MAE = 4
					прогноз = 7	MAE = 1
						CV = 3.00

```
KFold(n_splits=3,
      shuffle=False)
# CV(3) = 4.33

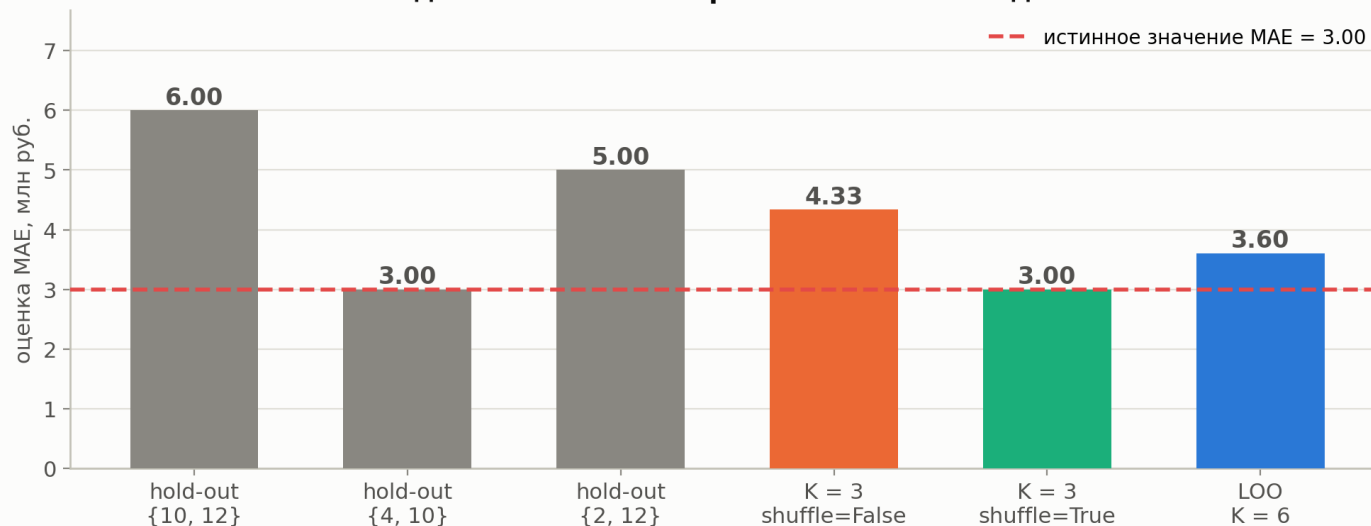
KFold(n_splits=3,
      shuffle=True,
      random_state=42)
# CV(3) = 3.00
```

- исходные данные упорядочены по значению целевой переменной;
- при shuffle=False каждый блок содержит однородные наблюдения, а обучающая выборка систематически смещена;
- значение по умолчанию — shuffle=False.

Перестановка наблюдений изменяет оценку с 4.33 на 3.00. Перемешивание необходимо, если данные упорядочены по времени, идентификатору или классу; исключения составляют временные ряды, где порядок наблюдений должен сохраняться.

Контроль по одному объекту и сверка расчёта

Рис. 11. Оценки MAE, полученные шестью схемами на одной и той же выборке из шести наблюдений



```
cross_val_score(  
    DummyRegressor('mean'), X, y,  
    cv=KFold(3, shuffle=False),  
    scoring='neg_mean_absolute_error')
```

```
# по блокам: [6. 1. 6.]  
# среднее: 4.3333  
# ручной расчёт: 4.3333
```

- истинное значение: прогноз 7, $MAE = (5+3+1+1+3+5)/6 = 3.00$;
- LOO ($K = 6$): оценка 3.60 при вдвое большем числе обучений;
- однократное разбиение: 3.00, 5.00 или 6.00 в зависимости от состава контрольной пары;
- оценки смещены вверх, так как модель обучается на 4–5 наблюдениях вместо 6.

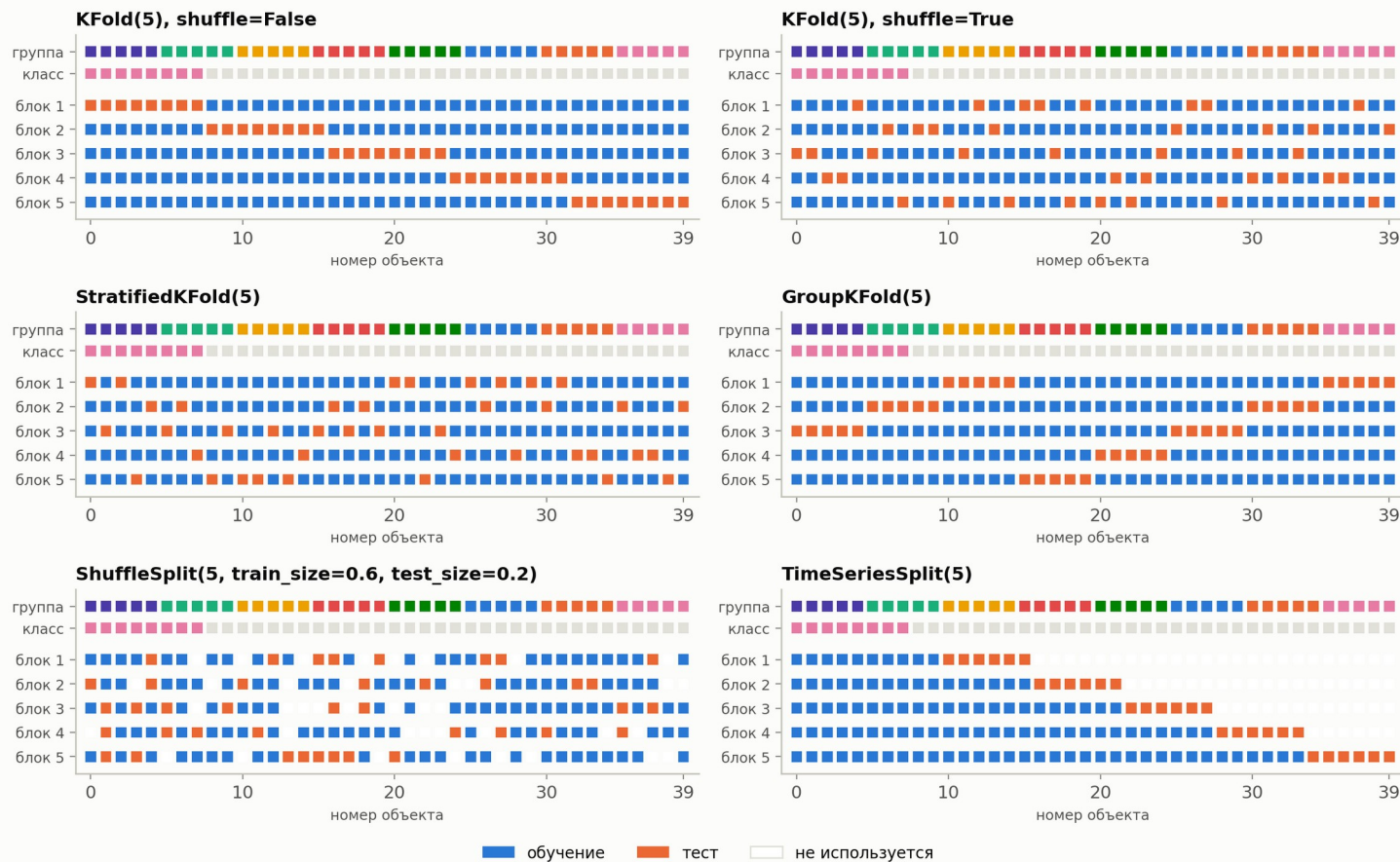
Пример иллюстрирует три свойства процедуры: вычислительная схема сводится к элементарным операциям; результат зависит от порядка наблюдений; на малой выборке однократное разбиение даёт оценку с недопустимо высокой дисперсией.

4

Альтернативные схемы и их сравнение

Схемы разбиения выборки

Рис. 12. Структура разбиений для шести схем кросс-валидации (n = 40)



```
from sklearn.model_selection import (  
    KFold, StratifiedKFold,  
    GroupKFold, StratifiedGroupKFold,  
    ShuffleSplit, LeaveOneOut,  
    RepeatedStratifiedKFold,  
    TimeSeriesSplit)  
  
for train, test in cv.split(X, y, groups):  
    ...
```

Критерий выбора схемы

наличие временного порядка

TimeSeriesSplit

наличие групп связанных объектов

GroupKFold

дисбаланс классов

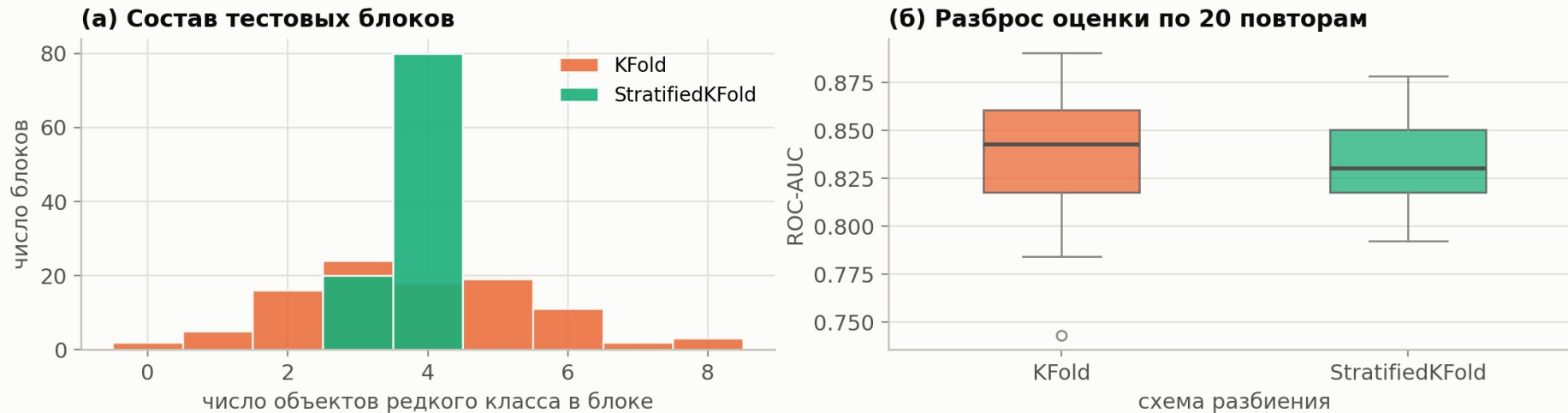
StratifiedKFold

перечисленные условия не выполнены

KFold(shuffle=True)

Стратифицированное разбиение

Рис. 13. Состав тестовых блоков и разброс оценки ROC-AUC при KFold и StratifiedKFold (доля редкого класса 6 %)



```
kf = KFold(n_splits=5, shuffle=True, random_state=seed)
sk = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed)

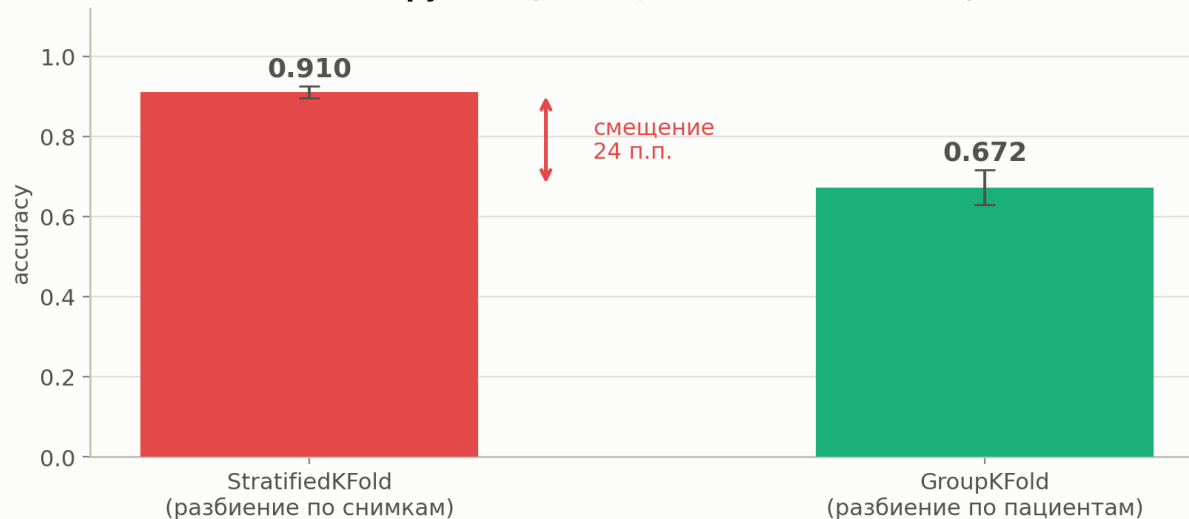
cross_val_score(model, X, y, cv=sk, scoring='roc_auc')
```

- KFold: от 0 до 8 объектов редкого класса в блоке
- StratifiedKFold: от 3 до 4
- в 2 блоках из 100 у KFold редкий класс отсутствовал, ROC-AUC не определён
- стандартное отклонение оценки выше в 1.6 раза

Выборка 300 объектов, доля редкого класса 6 %. При передаче классификатору параметра `cv=5` стратификация применяется по умолчанию; при явной передаче объекта KFold — не применяется.

Разбиение при наличии групп связанных объектов

Рис. 14. Оценка accuracy при разбиении по объектам и по группам (40 пациентов × 10 снимков)



```
groups = patient_id # длина как у y  
  
cross_val_score(model, X, y,  
                 groups=groups,  
                 cv=GroupKFold(n_splits=5))
```

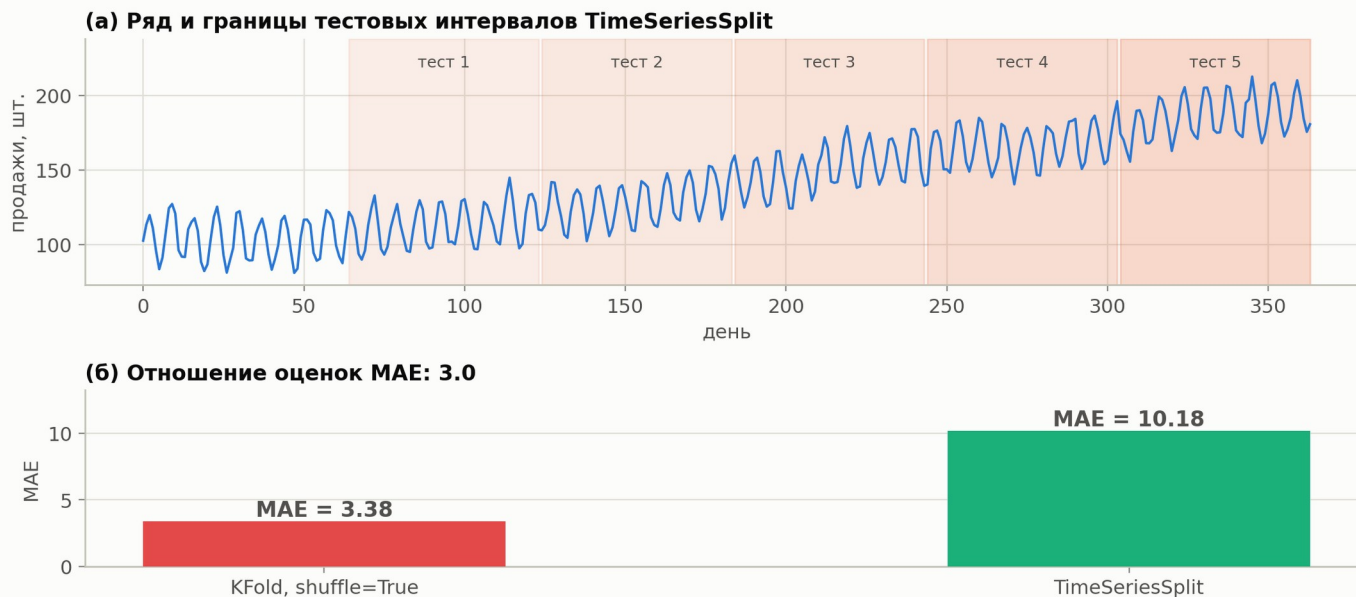
```
# StratifiedGroupKFold – стратификация  
# с сохранением целостности групп
```

- снимки одного пациента статистически зависимы;
- при разбиении по снимкам часть изображений пациента попадает в обучающую выборку, часть — в контрольную, и модель различает пациентов, а не классы;
- GroupKFold помещает группу целиком в один блок;
- аналогичные постановки: изображения одного объекта, записи одного диктора, сессии одного пользователя, измерения одного датчика.

Выборка: 40 пациентов по 10 снимков; целевая переменная определена на уровне пациента. Оценка accuracy составила 0.910 ± 0.015 при разбиении по снимкам и 0.672 ± 0.044 при разбиении по пациентам.

Хронологическое разбиение

Рис. 15. Схема TimeSeriesSplit и сравнение MAE при перемешанном и хронологическом разбиении



```
tscv = TimeSeriesSplit(n_splits=5, gap=0)
```

```
for train, test in tscv.split(X):  
    assert train.max() < test.min()
```

```
cross_val_score(model, X, y, cv=tscv,  
                 scoring='neg_mean_absolute_error')
```

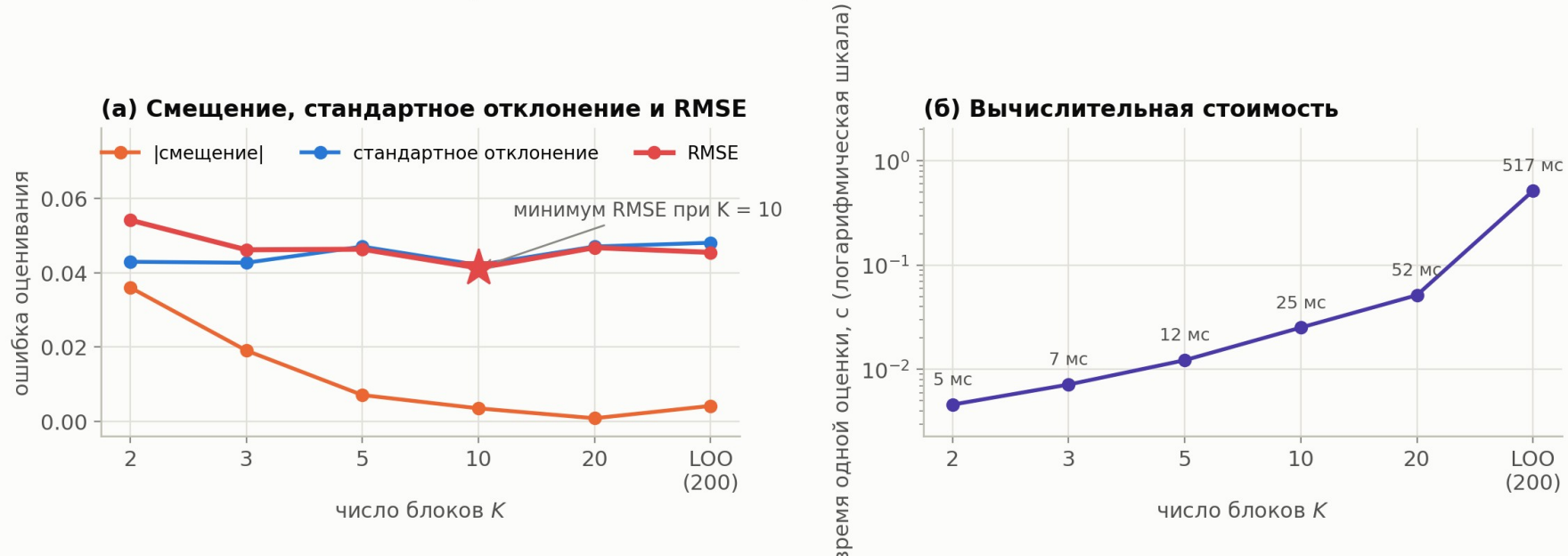
```
# gap задаёт зазор между обучающим  
# и контрольным интервалами
```

- при перемешивании наблюдение за день $t-1$ попадает в обучающую выборку, а за день t — в контрольную;
- решается задача интерполяции пропусков в известном прошлом, а не прогнозирования;
- деревья не экстраполируют тренд: на новых интервалах прогноз равен уровню последнего наблюдавшегося.

Ряд: ежедневные продажи за 364 дня, модель — случайный лес. MAE при KFold(shuffle=True) составила 3.38, при TimeSeriesSplit — 10.18.

Зависимость свойств оценки от числа блоков

Рис. 16. Зависимость смещения, стандартного отклонения, RMSE и времени вычисления оценки от числа блоков K



```
idx = rng.choice(POP_SIZE, 200, replace=False) # рабочая выборка
rest = np.setdiff1d(np.arange(POP_SIZE), idx) # остальные 19 800
truth = model().fit(X[idx], y[idx]).score(X[rest], y[rest])
est = cross_val_score(model(), X[idx], y[idx], cv=scheme()).mean()
bias.append(est - truth)
```

- $K = 2$: смещение -0.036
- $K = 5$: смещение -0.007
- $K = 10$: смещение -0.003 , минимум RMSE
- LOO: 200 обучений вместо 5

Генеральная совокупность 20 000 объектов, 60 повторов по 200 объектов, поэтому истинное значение известно. Смещение убывает с ростом K , дисперсия при этом не уменьшается, а вычислительная стоимость растёт линейно.

Сравнение схем оценивания

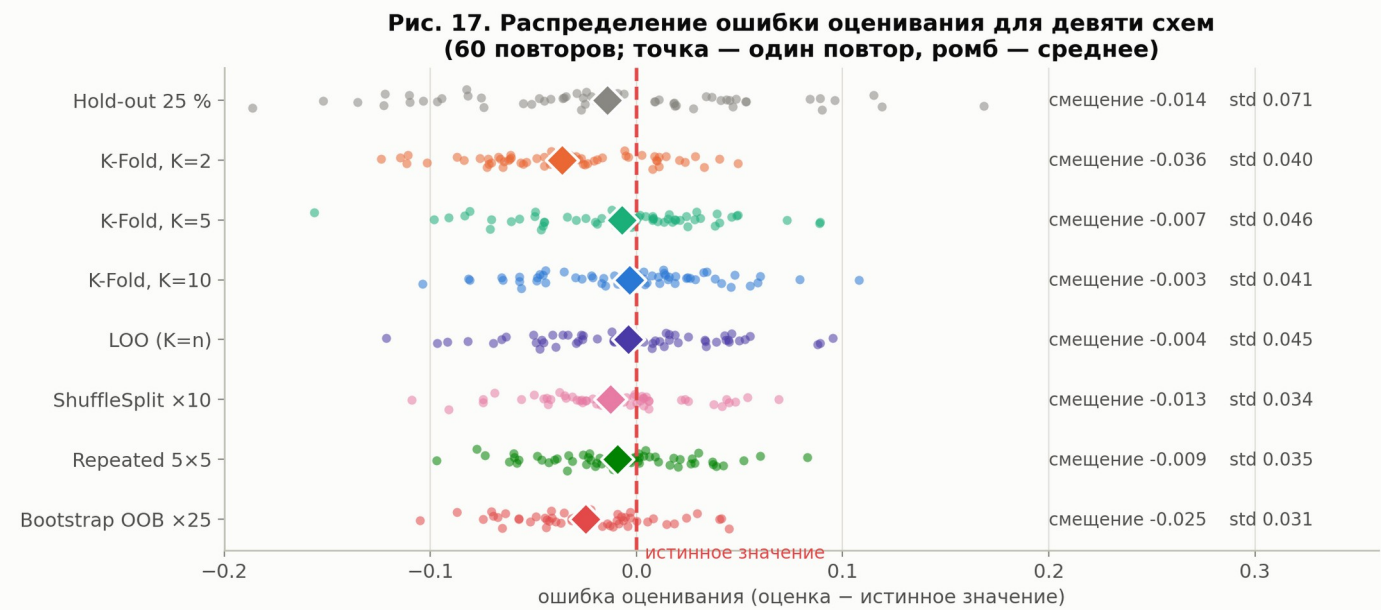


схема	смещ.	std	RMSE
Hold-out 25 %	-0.014	0.071	0.072
K-Fold, K = 2	-0.036	0.043	0.054
K-Fold, K = 3	-0.019	0.043	0.046
K-Fold, K = 5	-0.007	0.047	0.046
K-Fold, K = 10	-0.003	0.042	0.041
LOO (K = n)	-0.004	0.048	0.046
ShuffleSplit ×10	-0.013	0.034	0.036
Repeated 5×5	-0.009	0.035	0.037
Bootstrap OOB	-0.025	0.030	0.039

- однократное разбиение уступает по дисперсии в 1.5 раза при сопоставимом смещении;
- схемы с повторами усредняют по 25–50 разбиениям и дают наименьшую RMSE;
- оценка out-of-bag смещена вниз: обучающая выборка содержит около 63 % уникальных объектов.

Величина RMSE объединяет смещение и дисперсию и служит основным критерием сравнения схем.

Рекомендации по применению

Условие	Схема	Обоснование
Общий случай	KFold(5, shuffle=True, random_state=...)	минимальная суммарная ошибка при приемлемой стоимости
Классификация, дисбаланс классов	StratifiedKFold	сохраняет доли классов, исключает вырожденные блоки
Объекты образуют группы	GroupKFold, StratifiedGroupKFold	исключает одновременное попадание группы в обучающую и контрольную выборки
Данные упорядочены по времени	TimeSeriesSplit(gap=...)	обучение только на предшествующих наблюдениях
Требуется различить близкие модели	RepeatedStratifiedKFold	усреднение по повторам снижает дисперсию оценки
Малый объём выборки ($n < 100$)	LeaveOneOut, RepeatedKFold	минимальное смещение при допустимой стоимости
Большой объём или дорогое обучение	однократное разбиение	дисперсия оценки мала при больших контрольных выборках

```
# отчёт о качестве: среднее и стандартное отклонение по блокам
# подбор гиперпараметров: внутренний цикл; оценка качества: внешний цикл
# все обучаемые преобразования – внутри Pipeline
# random_state фиксируется во всех источниках случайности
```

Заключение

- 1 Кросс-валидация оценивает ошибку модели на данных, не участвовавших в её обучении; эмпирическая ошибка такой оценкой не является.
- 2 Результат процедуры — среднее и стандартное отклонение по блокам. Оценка по одному разбиению имеет дисперсию, превышающую типичные различия между моделями.
- 3 Преобразования, оценивающие параметры по данным, должны выполняться внутри процедуры; иначе оценка систематически смещается вверх.
- 4 Схема разбиения определяется структурой данных: временной порядок, группировка объектов, дисбаланс классов.
- 5 Значения $K = 5$ и $K = 10$ обеспечивают малое смещение при приемлемой вычислительной стоимости, что подтверждено измерением на известной генеральной совокупности.

Источники

Hastie T., Tibshirani R., Friedman J. The Elements of Statistical Learning. 2nd ed. Springer, 2009. Гл. 7. · Kohavi R. A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. IJCAI, 1995. · scikit-learn User Guide, § 3.1 Cross-validation.